

LAB SHEET 6

Experiment Name - > Predictive Analytics using Linear Regression and Model Performance Evaluation

- Steps :
1. Select a suitable dataset such as the House Price Prediction Dataset, Advertising Sales Dataset, or California Housing Dataset.
 2. Import the required libraries including Pandas, NumPy, Matplotlib, and Scikit-learn, and load the dataset.
 3. Perform basic data preprocessing by handling missing values, encoding categorical variables (if any), and selecting relevant features.
 4. Split the dataset into training and testing sets using an appropriate train-test ratio (e.g., 80:20).
 5. Train a Linear Regression model using the training dataset.
 6. Use the trained model to predict values for the testing dataset.
 7. Visualize the regression line and compare the actual and predicted values using appropriate plots.
 8. Evaluate the model using performance metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score.
 9. Analyze the regression coefficients to understand the impact of each independent variable on the target variable.
 10. Interpret the results and discuss the suitability of Linear Regression for the selected prediction problem.

Solutions ->

```
STEP 1 AND 2 : Import libraries and load the dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
df['MedHouseVal'] = housing.target
print(df.head())
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

STEP 3 : Data preprocessing

```
[3]: print(df.isnull().sum())
✓ Os df = df.fillna(df.median())
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']
print("Features:")
print(X.head())
print("\nTarget:")
print(y.head())
```

```
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude    0
MedHouseVal 0
dtype: int64
Features:
MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude \
0      8.3252    41.0      6.994127    1.023810      322.0      2.555556    37.88
1      8.3014    21.0      6.238137    0.971880     2401.0      2.109842    37.86
2      7.2574    52.0      8.288136    1.073446      496.0      2.802260    37.85
3      5.6431    52.0      5.817352    1.073059      558.0      2.547945    37.85
4      3.8462    52.0      6.281853    1.081081      565.0      2.181467    37.85

Longitude
0      -122.23
1      -122.22
2      -122.24
3      -122.25
4      -122.25
```

```
Target:
0      4.526
1      3.585
2      3.521
3      3.413
4      3.422
Name: MedHouseVal, dtype: float64
```

STEP 4 : Split the dataset into training and testing sets

```
[4]: X_train, X_test, y_train, y_test = train_test_split(
✓ Os X,
y,
test_size=0.20,
random_state=42
)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (16512, 8)
Testing data: (4128, 8)
```

STEP 5 : Train the Linear Regression model

```
[5]: model = LinearRegression()
✓ Os model.fit(X_train, y_train)
print("Linear Regression model trained successfully.")
```

Linear Regression model trained successfully.

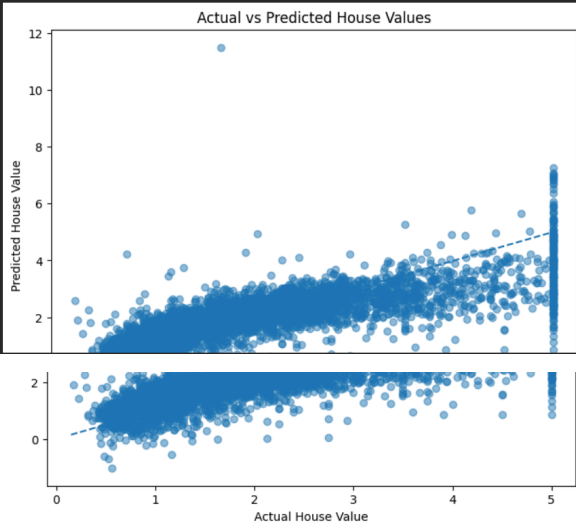
STEP 6 : Predict values for the testing dataset

```
[6]: y_pred = model.predict(X_test)
✓ Os comparison = pd.DataFrame({
'Actual Price': y_test.values,
'Predicted Price': y_pred
})
print(comparison.head(10))
sample_prediction = model.predict(X_test.iloc[[0]])
print("Predicted house value:", sample_prediction[0])
print("Actual house value:", y_test.iloc[0])
```

```
Actual Price  Predicted Price
0      0.47700      0.719123
1      0.45800      1.764017
2      5.00001      2.709659
3      2.18600      2.838926
4      2.78000      2.604657
5      1.50700      2.011754
6      1.98200      2.645500
7      1.57500      2.168755
8      3.40000      2.740746
9      4.46000      3.915615
Predicted house value: 0.7191228416019086
Actual house value: 0.477
```

STEP 7 : Visualize actual vs predicted values

```
[7] ✓ 0s  
plt.figure(figsize=(8, 6))  
plt.scatter(y_test, y_pred, alpha=0.5)  
plt.xlabel("Actual House Value")  
plt.ylabel("Predicted House Value")  
plt.title("Actual vs Predicted House Values")  
plt.plot(  
    [y_test.min(), y_test.max()],  
    [y_test.min(), y_test.max()],  
    linestyle='--'  
)  
plt.show()
```



STEP 8 : Evaluate the model

```
[8] ✓ 0s  
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
r2 = r2_score(y_test, y_pred)  
print("Mean Absolute Error (MAE):", mae)  
print("Mean Squared Error (MSE):", mse)  
print("Root Mean Squared Error (RMSE):", rmse)  
print("R² Score:", r2)  
  
Mean Absolute Error (MAE): 0.5332001304956553  
Mean Squared Error (MSE): 0.5558915986992444  
Root Mean Squared Error (RMSE): 0.7455813830127764  
R² Score: 0.5757877060324508
```

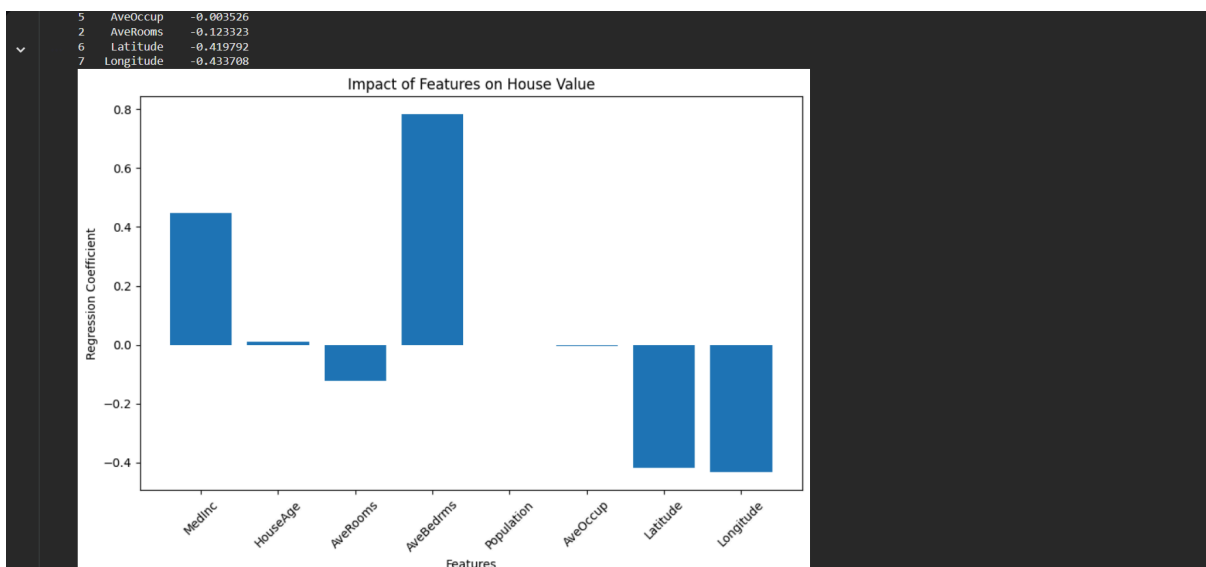
```

STEP 9: Analyze regression coefficients

[9] coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})
print(coefficients)
coefficients_sorted = coefficients.sort_values(
    by='Coefficient',
    ascending=False
)
print(coefficients_sorted)
plt.figure(figsize=(10, 6))
plt.bar(
    coefficients['Feature'],
    coefficients['Coefficient']
)
plt.xlabel("Features")
plt.ylabel("Regression Coefficient")
plt.title("Impact of Features on House Value")
plt.xticks(rotation=45)
plt.show()

```

Feature	Coefficient
MedInc	0.448675
HouseAge	0.009724
AveRooms	-0.123323
AveBedrms	0.783145
Population	-0.000002
AveOccup	-0.003526
Latitude	-0.419792
Longitude	-0.433708



Questions

1. What is Predictive Analytics, and how is it used in real-world applications?
2. Explain the working principle of the Linear Regression algorithm.
3. Differentiate between dependent and independent variables with suitable examples.
4. Why is it necessary to split the dataset into training and testing sets?
5. What is the significance of the R^2 Score in regression analysis?
6. Differentiate between MAE, MSE, and RMSE. Which metric is more sensitive to large prediction errors?
7. What assumptions should be satisfied before applying Linear Regression?
8. How can overfitting and underfitting affect the performance of a regression model?
9. Mention any three real-world applications of Linear Regression in business or industry.

10. How can feature selection improve the accuracy and interpretability of a predictive model?

Solution -> 1) **Predictive Analytics** is the process of using historical data, statistical techniques, and machine learning algorithms to predict future outcomes or events.

It analyzes patterns in past data and uses them to make predictions about what is likely to happen in the future.

Real-world applications:

- **Banking:** Predicting loan defaults and credit risk.
- **Healthcare:** Predicting diseases and patient outcomes.
- **E-commerce:** Predicting customer purchases and recommending products.
- **Business:** Forecasting sales and future demand.
- **Weather:** Predicting weather conditions.

Thus, predictive analytics helps organizations make better decisions and reduce risks.

2) **Linear Regression** is a supervised machine learning algorithm used to predict a continuous numerical value.

It assumes that there is a **linear relationship** between the independent variable(s) and the dependent variable.

For simple linear regression, the equation is:

$$\hat{y} = b_0 + b_1x$$

where:

- $\hat{y}$ = predicted value
- b_0 = intercept
- b_1 = slope/coefficient
- x = independent variable

The algorithm finds the best-fitting straight line by minimizing the difference between actual and predicted values. This is generally done using the **Least Squares Method**, which minimizes the sum of squared errors.

3)

Independent Variable

Dependent Variable

It is the input variable.

It is the output variable.

It is used to predict the outcome.

It is the value being predicted.

Usually represented by **X**.

Usually represented by **Y**.

Its value is used as a predictor.

Its value depends on the independent variable.

Example: To predict house price based on house size:

- **Independent variable (X):** House size in square feet
- **Dependent variable (Y):** House price

As the size of the house changes, its price may also change.

4) A dataset is divided into **training** and **testing** sets to evaluate how well a machine learning model performs on unseen data.

- **Training set:** Used to train and learn the patterns in the data.
- **Testing set:** Used to evaluate the trained model on data that it has not seen before.

A common split is **80% training and 20% testing**.

This separation helps determine whether the model has learned general patterns or has simply memorized the training data. It also helps identify **overfitting** and provides a more realistic estimate of model performance.

5) The **R² Score**, or **coefficient of determination**, measures how well a regression model explains the variation in the dependent variable.

Its value is generally interpreted as follows:

- **$R^2 = 1$** : Perfect prediction.
- **$R^2 = 0$** : The model does not explain the variation better than a baseline based on the mean.
- **Higher R^2** : The model explains more of the variation in the target variable.

For example, an **R^2 score of 0.80** means that approximately **80% of the variation** in the dependent variable is explained by the model, under the usual interpretation.

However, a high R^2 alone does not guarantee that the model is appropriate or that it will perform well on new data.

6)

Metric	Meaning	Characteristics
MAE	Mean Absolute Error	Average absolute difference between actual and predicted values
MSE	Mean Squared Error	Average of squared prediction errors
RMSE	Root Mean Squared Error	Square root of MSE

Formulas:

$$MAE = \frac{1}{n} \sum |y - \hat{y}| \quad MSE = \frac{1}{n} \sum (y - \hat{y})^2 \quad RMSE = \sqrt{MSE}$$

MSE and RMSE are more sensitive to large prediction errors because the errors are squared before averaging. MAE treats errors more linearly.

7) Some important assumptions of Linear Regression are:

1. **Linearity:** There should be a linear relationship between the predictors and the dependent variable.
2. **Independence:** Observations and errors should generally be independent.
3. **Homoscedasticity:** The variance of errors should remain approximately constant across predicted values.
4. **Normality of residuals:** Residuals should be approximately normally distributed, particularly for reliable statistical inference.
5. **No significant multicollinearity:** Independent variables should not be highly correlated with each other.
6. **No major influential outliers:** Extreme observations should not disproportionately affect the regression model.

Checking these assumptions helps ensure that the regression results are reliable and interpretable.

8) **Overfitting** occurs when a model learns the training data too closely, including noise and random variations.

- Very good performance on training data.
- Poor performance on unseen testing data.
- Usually indicates **high variance**.

Underfitting occurs when the model is too simple to capture the important patterns in the data.

- Poor performance on training data.
- Poor performance on testing data.
- Usually indicates **high bias**.

The goal is to build a model that achieves a good balance between **underfitting and overfitting** and generalizes well to unseen data.

9) 1. **Sales Forecasting:**

Businesses can predict future sales based on factors such as advertising expenditure, previous sales, and market conditions.

2. **House Price Prediction:**

Real-estate companies can estimate property prices using features such as area, number of rooms, location-related variables, and age of the property.

3. Demand Forecasting:

Companies can predict the future demand for products based on historical demand, price, season, and other relevant factors.

Other applications include salary prediction, revenue forecasting, energy consumption prediction, and estimating customer spending.

10) **Feature selection** is the process of selecting the most relevant input variables for a machine learning model and removing unnecessary or redundant features.

It can improve a model in several ways:

1. **Reduces noise:** Irrelevant features can introduce noise into the model.
2. **Reduces overfitting:** Using fewer relevant features can make the model less likely to memorize noise.
3. **Improves performance:** Removing unnecessary variables can make the model more efficient and sometimes improve prediction accuracy.
4. **Improves interpretability:** A model with fewer important features is easier to understand and explain.
5. **Reduces computational cost:** Fewer features require less processing and storage.

For example, while predicting house prices, features such as **house area, number of rooms, and location-related variables** may be useful, while an irrelevant ID number should be removed.